

Ứng dụng RAG trong việc hỏi đáp tài liệu bài học AIO

Quoc-Thai Nguyen, Quang-Hien Ho, Quang-Vinh Dinh

I. Giới thiệu

Large Language Models (LLMs) (Tạm dịch: Các mô hình ngôn ngữ lớn) là loại mô hình cho phép người dùng nhập vào một văn bản với nội dung bất kì, thường sẽ là các yêu cầu hoặc câu hỏi. Từ đó, mô hình sẽ trả về câu trả lời dưới dạng văn bản thoả mãn yêu cầu của người dùng. Các ứng dụng phổ biến nhất về LLM có thể kể đến như ChatGPT, Gemini...

II. Bài toán Object Detection và các phiên bản YOLO đời trước

III. Bài toán Object Detection

Trong Computer Vision, bài toán Object Detection hướng đến xây dựng một chương trình có thể tự động xác định vị trí và nhận diện tên (class) của các vật thể trong một bức ảnh. Tổng hợp hai thông tin đầu ra này còn được gọi với tên là bounding box. Về đây, ta có thể mô tả Input/Output của một chương trình Object Detection như sau:

- **Input:** Một bức ảnh.
- **Output:** Bounding box của các vật thể cần phát hiện trong ảnh.



Hình 3 Minh họa Input/Output của bài toán Object Detection.

Để thực hiện bài toán, các phương pháp sử dụng mạng Deep Learning cho thấy hiệu suất vượt trội. Ta có thể tìm tài liệu hướng tiếp cận theo ba dạng như sau:

1. **One-stage Object Detection:** Việc xác định vị trí tọa độ và phân loại tên class của các vật thể được thực hiện trên một bước duy nhất. Điển hình cho hướng tiếp cận này có thể kể đến SSD [1] và YOLO [1, 2, 3, 4, 5, 6, 7, 8, 9, 10].
2. **Two-stage Object Detection:** Việc xác định vị trí tọa độ và phân loại tên class của các vật thể được thực hiện riêng biệt. Điển hình cho hướng tiếp cận này có thể kể đến RCNN [11] và Faster RCNN [12].
3. **End-to-end Object Detection:** Việc xác định vị trí tọa độ và phân loại tên class của các vật thể được dự đoán bởi một mô hình duy nhất (thông lệ dùng các bước tiền và hậu xử lý bounding box). Điển hình cho hướng tiếp cận này có thể kể đến DETR [13], DINO [14], và DeFCN [15].

Question: Cho tôi danh sách một số phương pháp Object Detection theo hướng tiếp cận End-to-end Object Detection?

Helpful Answer:

• DETR [16]

• DINO [17]

• DeFCN [18] Sources: source_0, source_1, source_2

Câu trả lời

source_0

Detection [10]. Với những cải tiến mới, mô hình đã đạt được hiệu suất vượt trội hơn so với các phiên bản YOLO trước đó ở các khía cạnh khác nhau, tăng cường khả năng phát hiện đối tượng theo thời gian thực (real-time object detection).

1

Vùng thông tin truy vấn được

source_1

kể đến SSD [13] và YOLO [1, 2, 3, 4, 5, 6, 7, 8, 9, 10].

2. Two-stage Object Detection: Việc xác định vị trí tọa độ và phân loại tên class của các vật thể được thực hiện riêng biệt. Điển hình cho hướng tiếp cận này có thể kể đến RCNN [14] và Faster RCNN [15].

3. End-to-end Object Detection: Việc xác định vị trí tọa độ và phân loại tên class của các vật thể được dự đoán bởi một mô hình duy nhất (không sử dụng các bước tiền và hậu xử lý bounding box). Điển hình cho hướng tiếp cận này có thể kể đến DETR [16], DINO [17], và DeFCN [18].

5

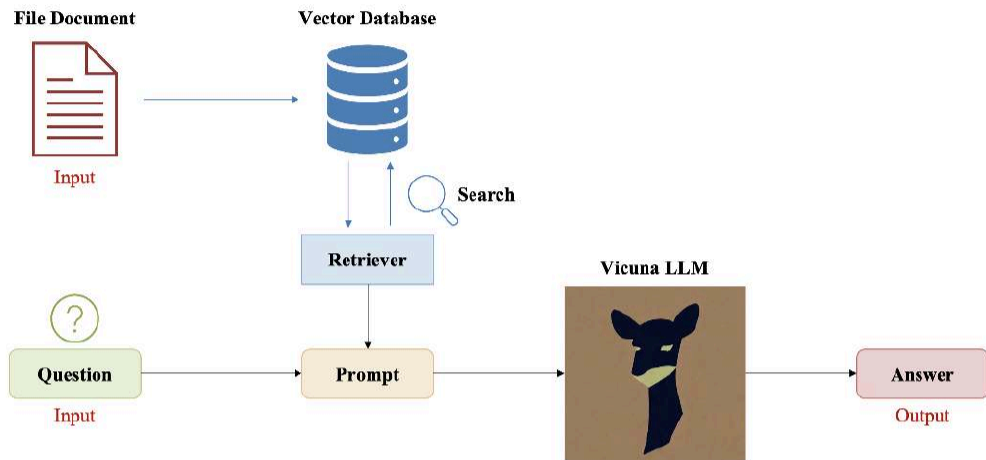
Hình 1: Sử dụng RAG trong việc hỏi đáp tài liệu về YOLOv10 trong AIO.

Trong LLMs, **Retrieval Augmented Generation (RAG)** là một kỹ thuật giúp LLM cải thiện chất lượng kết quả trả lời bằng cách tích hợp nội dung truy vấn được tìm từ nguồn tài liệu nào đó để trả lời cho một câu hỏi đầu vào. Trong project này, chúng ta sẽ tìm hiểu cách xây dựng một chương trình RAG cơ bản. Đồng thời, ứng dụng chương trình vào việc hỏi đáp tài liệu bài học trong khóa AIO. Theo đó, Input và Output của chương trình là:

1

- **Input:** File tài liệu cần hỏi đáp và một câu hỏi liên quan đến nội dung tài liệu.
- **Output:** Câu trả lời.

Tổng quan, luồng xử lý (pipeline) của chương trình RAG mà chúng ta sẽ xây dựng có dạng như sau:



Hình 2: Pipeline của chương trình RAG trong project.

Mục lục

I.	Giới thiệu	1
II.	Xây dựng chương trình	4
III.	Xây dựng giao diện	11
III.1.	Cài đặt môi trường conda	11
III.2.	Cài đặt thư viện cần thiết	11
III.3.	Định nghĩa các hàm hỗ trợ	13
III.4.	Xây dựng giao diện người dùng	15
III.5.	Chạy ứng dụng và demo từng bước	17
IV.	Câu hỏi trắc nghiệm	20
V.	Tài liệu tham khảo	22
	Phụ lục	23

II. Xây dựng chương trình

II.1. Chương trình RAG

Trước tiên, chúng ta cần nắm được các bước xử lý cơ bản trong RAG bằng cách xây dựng một chương trình RAG đơn giản. Tại đây, ta sẽ sử dụng thư viện LangChain để thực hiện việc này. Các bước triển khai như sau:

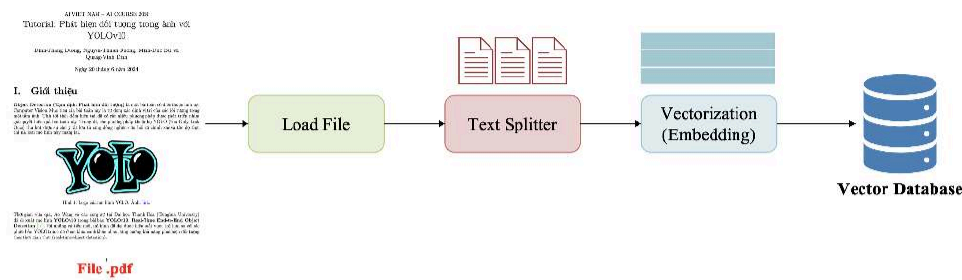


Hình 3: Thư viện LangChain. Một thư viện được thiết kế chuyên biệt cho việc xây dựng các ứng dụng về LLMs. Ảnh: link.

1. **Cài đặt các gói thư viện cần thiết:** Thực hiện cài đặt một số gói thư viện thông qua lệnh pip install như sau:

```
1 !pip install qtransformers==4.52.4
2 !pip install qbitsandbytes==0.46.0
3 !pip install qaccelerate==1.7.0
4 !pip install qlangchain==0.3.25
5 !pip install qlangchainhub==0.1.21
6 !pip install qlangchain-chroma==0.2.4
7 !pip install qlangchain_experimental==0.3.4
8 !pip install qlangchain-community==0.3.24
9 !pip install qlangchain_huggingface==0.2.0
10 !pip install qpython-dotenv==1.1.0
11 !pip install qpypdf
```

2. **Xây dựng vector database:** Để thực hiện truy vấn, chúng ta cần phải có một cơ sở dữ liệu. Theo `humoidungproject`, với dữ liệu nguồn là một file pdf, chúng ta sẽ thực hiện đưa các nội dung trong file này vào cơ sở dữ liệu. Về các bước thực hiện, các bạn có thể coi qua ảnh sau:



Hình4:Pipelinecác bước thực hiện xây dựng vectordatabase.

(a)Import các thư viện cần thiết:Để xây dựng vectordatabase,chúng ta cần một số thư viện sau:

```
1import torch
2
3from transformersimport BitsAndBytesConfig
4from transformersimport AutoTokenizer, AutoModelForCausalLM, pipeline
5from langchain_huggingfaceimport HuggingFaceEmbeddings
6from langchain_huggingface.llmsimport HuggingFacePipeline
7
8from langchain.memoryimport ConversationBufferMemory
9from langchain_community.chat_message_historiesimport ChatMessageHistory
10from langchain_community.document_loadersimport PyPDFLoader, TextLoader
11from langchain.chainsimport ConversationalRetrievalChain
12from langchain_experimental.text_splitterimport SemanticChunker
13
14from langchain_chromaimport Chroma
15from langchain_text_splittersimport RecursiveCharacterTextSplitter
16from langchain_core.runnablesimport RunnablePassthrough
17from langchain_core.output_parsersimport StrOutputParser
18from langchainimport hub
```

(b)Đọc file pdf:Tìm một file pdf cho trước (trong bài này ta sẽ sử dụng file mô tả là bài hướng dẫn YOLOv10, các bạn có thể tải file này tại đây), ta sử dụng class PyPDFLoader để đọc file pdf này lên như sau:

```
1Loader=PyPDFLoader
2FILE_PATH="./AIO-2024-All-Materials.pdf"
3loader=Loader(FILE_PATH)
4documents=loader.load()
```

(c)Khởi tạo mô hình embedding:Để chuyển đổi các văn bản thành vector, ta sử dụng mô hình embedding. Mô hình này giúp biểu diễn văn bản dưới dạng các vector, từ đó giúp việc truy vấn và tìm kiếm thông tin trở nên chính xác hơn.

Trong ví dụ này, chúng ta sử dụng mô hình `HuggingFaceEmbeddings` với mô hình `bkai-foundation-models/vietnamese-bi-encoder`, một mô hình được chuẩn luyện để xử lý ngôn ngữ tiếng Việt.

Chúng ta tạo embedding model như sau:

```
1 embeddings = HuggingFaceEmbeddings(  
2     model_name="bkai-foundation-models/vietnamese-bi-encoder"  
3 )
```

Mô hình `embedding` này sẽ giúp chuyển các văn bản thành các vector. Các vector này sẽ được sử dụng trong quá trình **semantic chunking** để chia văn bản thành các phần nhỏ có ý nghĩa hơn, thay vì tách theo độ dài cố định.

(d) **Khởi tạo bộ tách văn bản (text splitter):** Trước đây, khi sử dụng phương pháp phân chia văn bản đơn giản, văn bản thường bị tách thành các phần nhỏ đồng đều mà không xem xét ngữ nghĩa. Tuy nhiên, để cải thiện chất lượng của các đoạn văn bản nhỏ, chúng ta chuyển sang sử dụng `SemanticChunker`, một kỹ thuật giúp tách văn bản dựa trên ngữ nghĩa, thay vì chỉ dựa vào độ dài đoạn văn.

```
1 semantic_splitter = SemanticChunker(  
2     embeddings=embeddings,  
3     buffer_size=1,  
4     breakpoint_threshold_type="percentile",  
5     breakpoint_threshold_amount=95,  
6     min_chunk_size=500,  
7     add_start_index=True  
8 )
```

Semantic Chunking giúp chia nhỏ văn bản thành các phần có ý nghĩa, giúp tăng tính chính xác khi thực hiện truy vấn. Các tham số trong `SemanticChunker` giúp điều chỉnh cách thức và cách tách các đoạn văn bản này:

- **buffer_size=1:** Xác định số câu cần gom lại trước khi tách. Ở đây, giá trị `buffer_size=1` có nghĩa là mỗi nhóm bao gồm một câu, giúp mỗi chunk chứa các câu riêng biệt.
- **breakpoint_threshold_type="percentile":** Tham số này chỉ định cách tính điểm phân đoạn. Trong trường hợp này, `percentile` có nghĩa là dựa trên tỷ lệ phần trăm của sự khác biệt về ngữ nghĩa giữa các đoạn văn. Tức là, nếu độ tương đồng của hai đoạn văn thấp hơn ngưỡng nhất định, chúng sẽ được tách ra làm các chunk riêng biệt.
- **breakpoint_threshold_amount=95:** Đây là ngưỡng phần trăm dùng để xác định khi nào nên cắt các đoạn văn bản. Giá trị 95 có nghĩa là nếu độ tương đồng giữa hai đoạn văn thấp hơn 95%, chúng sẽ được tách rời, tạo thành các chunk mới. Điều này đảm bảo rằng các đoạn văn bản không bị cắt ngang một cách ngẫu nhiên mà vẫn giữ được sự liên kết ngữ nghĩa.
- **min_chunk_size=500:** Thiết lập kích thước tối thiểu của mỗi chunk. Khi giá trị này được đặt thành 500, mỗi đoạn văn bản sẽ có ít nhất 500 ký tự. Điều này đảm bảo

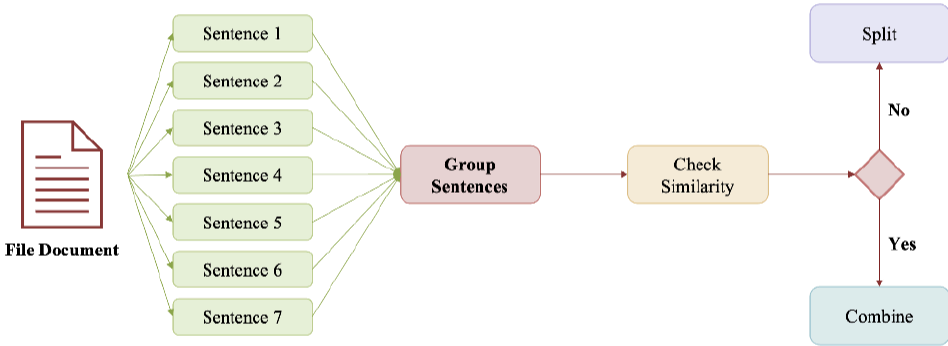
rằng các chunk không quá ngắn và vẫn chứa đủ thông tin để có thể sử dụng hiệu quả trong các tác vụ tìm kiếm hoặc truy vấn.

- **add_start_index=True:** Tham số này giúp đánh dấu vị trí của mỗi chunk trong văn bản gốc, tạo chỉ mục cho mỗi phần, giúp dễ dàng truy cập và tổ chức các chunk sau khi đã được tách ra.

Với cách tham số này, **SemanticChunker** giúp chia văn bản thành các phần có ý nghĩa và bảo đảm sự liên kết ngữ nghĩa trong mỗi chunk. Đoạn mã sau sẽ thực hiện quá trình này:

```
docs=semantic_splitter.split_documents(documents)
print("Number of semantic chunks: ", len(docs))
```

Kết quả của đoạn mã trên sẽ cho thấy số lượng chunk được tạo ra từ văn bản ban đầu, đồng thời mỗi chunk sẽ chứa một phần ngữ nghĩa đầy đủ từ tài liệu gốc.



Hình 5: Minh họa quy trình Semantic Chunking.

- (e) **Khởi tạo vector database:** Sau khi tách văn bản thành các chunk có ý nghĩa và biểu diễn chúng dưới dạng các vector với mô hình embedding, bước tiếp theo là khởi tạo một vector database để lưu trữ các vector của các chunk này. Ta sử dụng **Chroma** để khởi tạo vector database từ các documents đã được chia thành chunk và được vector hóa.

Mã thực hiện như sau:

```
vector_db=Chroma.from_documents(documents=docs,
                                embedding=embedding)
retriever=vector_db.as_retriever()
```

Kết quả của đoạn mã này là chúng ta đã tạo ra một vector database (**vector_db**) từ

các chunk đã được vector hóa, và khi tạo một retriever để có thể thực hiện các truy vấn từ cơ sở dữ liệu này.
Ta có thể thử thực hiện truy vấn với một đoạn văn bản bất kỳ tại đây:

```
1 result=retriever.invoke("What is YOLO?")
2
3 print("Number of relevant documents: ",len(result))
```

Kết quả của đoạn code này sẽ trả về cho ta danh sách các tài liệu có liên quan đến đoạn văn bản đầu vào.

3.**Khởi tạo mô hình ngôn ngữ lớn:** Trong project này, ta sẽ sử dụng mô hình Vicuna, một mô hình ngôn ngữ lớn nguồn mở có hiệu suất rất ổn, có thể phản hồi tốt với tiếng Việt. Các bạn có thể đọc thêm về mô hình Vicuna tại đây.



Hình 6: Mô hình Vicuna. Ảnh: link.

Một vấn đề của mô hình ngôn ngữ lớn đó là nó yêu cầu tài nguyên về phần cứng khá lớn. Vì vậy, để khởi tạo được mô hình Vicuna trên Colab, chúng ta sẽ cần thực hiện một số bước sau:

(a) **Khai báo một số cài đặt cần thiết cho mô hình:**

```
1 nf4_config=BitsAndBytesConfig(
2     load_in_4bit=True,
3     bnb_4bit_quant_type="nf4",
4     bnb_4bit_use_double_quant=True,
5     bnb_4bit_compute_dtype=torch.bfloat16
6 )
7
```

(b) Khởi tạo mô hình và tokenizer:

```
1 MODEL_NAME="lmsys/vicuna-7b-v1.5"
2
3 model=AutoModelForCausalLM.from_pretrained(
4     MODEL_NAME,
5     quantization_config=nf4_config,
6     low_cpu_mem_usage=True
7 )
8
9 tokenizer=AutoTokenizer.from_pretrained(model_name)
10
```

(c) Tích hợp tokenizer và model thành một pipeline để tiện sử dụng:

```
1 model_pipeline=pipeline(
2     "text-generation",
3     model=model,
4     tokenizer=tokenizer,
5     max_new_tokens=512,
6     pad_token_id=tokenizer.eos_token_id,
7     device_map="auto"
8 )
9
10 llm=HuggingFacePipeline(
11     pipeline=model_pipeline,
12 )
13
```

4. **Chạy chương trình:** Với vector database, retriever và mô hình Vicuna đã hoàn thiện. Ta sẽ kết hợp chúng lại để hoàn thành chương trình RAG đầu tiên của mình. Các bạn có thể test thử bằng cách đặt các câu hỏi có liên quan đến file tài liệu, câu trả lời của mô hình sẽ nằm ở phần **"Answer:"** trong output.

```
1 prompt=hub.pull("rlm/rag-prompt")
2
3 def format_docs(docs):
4     return "\n\n".join(doc.page_content for doc in docs)
5
6 rag_chain=(
7     {"context": retriever|format_docs, "question": RunnablePassthrough()}
8     | prompt
9     | llm
10    | StrOutputParser()
11 )
12
13 USER_QUESTION="YOLOv10 là gì?"
14 output=rag_chain.invoke(USER_QUESTION)
15 answer=output.split('Answer:')[1].strip()
```

```
16 print(answer)
```

```
Human: You are an assistant for question-answering tasks. Use the following pieces of retrieved context
to answer the question. If you know the answer to the question, provide a concise answer. If you do not
know the answer, say "I do not know".
Context: Quan sát kết quả trên, ta có thể thấy trên cùng một phiên bản, YOLOv10 có mức độ tối ưu tốt
hơn về mặt tham số mô hình cũng như độ trễ trong inference trong khi vẫn giữ được độ chính
xác ngang hoặc hơn so với các phiên bản trước.
18

khắc.
-Uu điểm : Cân bằng tốt giữa tốc độ và độ chính xác, dễ dàng sử dụng và triển khai.
-Nhược điểm : Yêu cầu phần cứng mạnh để đạt hiệu năng tối ưu.
5

AI VIETNAM (AIO2024) aivietnam.edu.vn
Hình 8: PSI và các kiến trúc tương tự. Ảnh: [9].
-Điểm mới : YOLOv9 sử dụng PSI và GELAN để cải thiện độ chính xác và hiệu suất của
mô hình.
-Uu điểm :
+ Kiến trúc tiên tiến: Sử dụng PSI và GELAN giúp mô hình duy trì thông tin quan
trọng và tối ưu hóa quá trình huấn luyện, làm cho YOLOv9 trở nên mạnh mẽ và linh
hoạt hơn trong nhiều ứng dụng khác nhau.
+ Tốc độ nhanh hơn: YOLOv9 có thể xử lý hình ảnh nhanh hơn so với YOLOv8 nhờ
vào các cải tiến trong kiến trúc mạng.
-Nhược điểm :
+ Mặc dù nhanh hơn YOLOv8, YOLOv9 vẫn yêu cầu nhiều tài nguyên tính toán, đặc
biệt là khi xử lý hình ảnh độ phân giải cao.
...
phiên bản YOLO trước đó ở các khía cạnh khác nhau, tăng cường khả năng phát hiện đối tượng
theo thời gian thực (real-time object detection).
1
Answer: YOLO là một hệ thống mô hình máy học được sử dụng cho việc phát hiện đối tượng trong hình ảnh.
```

Hình7:Minhhọakếtquảđầuraciủachươngtrình.

Như vậy, chúng ta đã hoàn thành một chương trình Python về RAG, có khả năng trả lời các câu hỏi dựa trên nội dung trong một tệp pdf.

III. Xây dựng giao diện

Sau khi có chương trình RAG có thể trả về output đúng như dự định, ta có thể tích hợp vào với một giao diện chat để có được một ứng dụng chat hoàn chỉnh. Để xây dựng phần giao diện trong project này, chúng ta sẽ sử dụng thư viện streamlit.



Hình 8: Streamlit, một thư viện hỗ trợ tạo ứng dụng web nhanh chóng cho Python.

III.1. Cài đặt môi trường conda

Để xây dựng ứng dụng Streamlit một cách dễ dàng và tránh xung đột thư viện, ta nên tạo một môi trường ảo Conda.

```
conda create -n aio-rag python=3.11
```

Trong lệnh trên, các tham số được giải thích như sau:

- **conda create**: Lệnh dùng để tạo một môi trường ảo mới.
- **-n aio-rag**: Dùng để đặt tên cho môi trường ảo là **aio-rag**. Bạn có thể thay đổi tên này.
- **python=3.11**: Chỉ định phiên bản Python sẽ được cài đặt trong môi trường này là 3.11.

Kích hoạt môi trường:

```
conda activate aio-rag
```

III.2. Cài đặt thư viện cần thiết

Sau khi kích hoạt môi trường, ta cài đặt các môi trường sau:

```
pip install transformers==4.40.0
pip install langchain==0.1.20
pip install langchainhub==0.1.15
pip install langchain-chroma==0.1.8
pip install langchain_experimental==0.0.61
pip install langchain-community==0.0.38
pip install langchain_huggingface==0.0.3
```

```
pipinstallpython-dotenv==1.0.0
pipinstallpypdf
pipinstallstreamlit==1.36.0
```

Dưới đây là một bản thảo về vai trò của từng nhóm thư viện trong dự án:

- **streamlit**: Xây dựng giao diện ứng dụng web tương tác.
- **transformers**: Load các mô hình LLM, tokenizer, pipeline.
- **langchain** (và các gói liên quan như **langchainhub**, **langchain-chroma**, **langchain_experimental**, **langchain-community**, **langchain_huggingface**): Bộ framework để xây dựng các ứng dụng dùng LLM, bao gồm kết nối dữ liệu (PDF), lưu trữ vector, và tích hợp các mô hình HuggingFace.
- **python-dotenv**: Quản lý các biến môi trường từ file `.env`.
- **pypdf**: Xử lý các file PDF (ví dụ: trích xuất văn bản).

Sau đó, tạo file code python và bắt đầu import các module cần thiết:

```
1 import streamlit as st
2 import tempfile
3 import os
4 import torch
5 from langchain_community.document_loaders import PyPDFLoader
6 from langchain_huggingface.embeddings import HuggingFaceEmbeddings
7 from langchain_experimental.text_splitter import SemanticChunker
8 from langchain_chroma import Chroma
9 from langchain_huggingface.llms import HuggingFacePipeline
10 from langchain import hub
11 from langchain_core.output_parsers import StrOutputParser
12 from langchain_core.runnables import RunnablePassthrough
13 from transformers import AutoModelForCausalLM, AutoTokenizer, pipeline
```

III.3. Khởi tạo Session State

Để đảm bảo ứng dụng Streamlit hoạt động ổn định và không bị tải lại models mỗi lần tương tác, ta cần khởi tạo các session state variables:

```
1 if "rag_chain" not in st.session_state:
2     st.session_state.rag_chain = None
3 if "models_loaded" not in st.session_state:
4     st.session_state.models_loaded = False
5 if "embeddings" not in st.session_state:
6     st.session_state.embeddings = None
7 if "llm" not in st.session_state:
8     st.session_state.llm = None
```

Cácsessionstatedượckhôitạobaogồm:

- rag_chain: LưutrữRAGchainđãđượcxây dựng từPDF.
- models_loaded: Kiểmtraxemcácmodelsđãđược tải haychưa.
- embeddings: Lưutrữembeddingmodelđãđược tải.
- llm: LưutrữLargeLanguageModelđãđược tải.

III.4. Địnhnghĩacáchhànhhỗ trợ

III.4.1. Hàm tảiEmbeddingModel

Tasử dụngdecorator@st.cache_resource đểcachemodel embeddings, tránh hviệctải lại mỗi lầnchạy:

```
1@st.cache_resource
2def load_embeddings():
3    return HuggingFaceEmbeddings(model_name="bkai-foundation-models/vietnamese-bi-
                                     encoder")
```

Hàmnàysử dụngmodel vietnamese-bi-encoder từBKAI FoundationModels, đượcổitừtiếngViệt.

III.4.2. Hàm tảiLargeLanguageModel

```
1@st.cache_resource
2def load_llm():
3    MODEL_NAME="lmsys/vicuna-7b-v1.5"
4    model=AutoModelForCausalLM.from_pretrained(
5        MODEL_NAME,
6        torch_dtype=torch.bfloat16,
7        low_cpu_mem_usage=True
8    )
9    tokenizer=AutoTokenizer.from_pretrained(MODEL_NAME)
10    model_pipeline=pipeline(
11        "text-generation",
12        model=model,
13        tokenizer=tokenizer,
14        max_new_tokens=512,
15        pad_token_id=tokenizer.eos_token_id,
16        device_map="auto"
17    )
18    return HuggingFacePipeline(pipeline=model_pipeline)
```

Tronghàmnày:

- MODEL_NAME: Sử dụngmodel Vicuna7Bv1.5, mộtmodel mạnh mẽ cho generation tasks.

- `torch_dtype=torch.bfloat16`: Sử dụng `bfloat16` để tiết kiệm memory.
- `low_cpu_mem_usage=True`: Tối ưu hóa việc sử dụng CPU memory.
- `max_new_tokens=512`: Giới hạn độ dài output tối đa là 512 tokens.
- `device_map="auto"`: Tự động phân bổ model lên GPU/CPU phù hợp.

III.4.3. Hàm xử lý PDF

```

1 def process_pdf(uploaded_file):
2     with tempfile.NamedTemporaryFile(delete=False, suffix=".pdf") as tmp_file:
3         tmp_file.write(uploaded_file.getvalue())
4         tmp_file_path = tmp_file.name
5
6     loader = PyPDFLoader(tmp_file_path)
7     documents = loader.load()
8
9     semantic_splitter = SemanticChunker(
10         embeddings=st.session_state.embeddings,
11         buffer_size=1,
12         breakpoint_threshold_type="percentile",
13         breakpoint_threshold_amount=95,
14         min_chunk_size=500,
15         add_start_index=True
16     )
17
18     docs = semantic_splitter.split_documents(documents)
19     vector_db = Chroma.from_documents(documents=docs, embedding=st.session_state.
                                     embeddings)
20     retriever = vector_db.as_retriever()
21
22     prompt = hub.pull("rlm/rag-prompt")
23
24     def format_docs(docs):
25         return "\n\n".join(doc.page_content for doc in docs)
26
27     rag_chain = (
28         {"context": retriever | format_docs, "question": RunnablePassthrough()}
29         | prompt
30         | st.session_state.llm
31         | StrOutputParser()
32     )
33
34     os.unlink(tmp_file_path)
35     return rag_chain, len(docs)

```

Hàm `process_pdf` thực hiện các bước sau:

- **Lưu file tạm**: Upload file từ Streamlit được chừa vào temporary file.
- **Load PDF**: Sử dụng `PyPDFLoader` để đọc nội dung PDF.

- **SemanticChunking:** Chia tài liệu thành các chunks dựa trên ngữ nghĩa với các tham số:
 - buffer_size=1: Kích thước buffer cho semantic chunking.
 - breakpoint_threshold_type="percentile": Sử dụng percentile để xác định breakpoint.
 - breakpoint_threshold_amount=95: Ngưỡng 95th percentile.
 - min_chunk_size=500: Kích thước chunk tối thiểu 500 ký tự.
- **Tạo VectorDatabase:** Sử dụng Chroma để lưu trữ embeddings của các chunks.
- **Xây dựng RAG Chain:** Kết hợp retriever, prompt template, và LLM thành một pipeline hoàn chỉnh.
- **Dọn dẹp:** Xóa temporary files sau khi xử lý xong.

III.5. Xây dựng giao diện người dùng

III.5.1. Cấu hình trang và tiêu đề

```
1st.set_page_config(page_title="PDFRAGAssistant",layout="wide")
2st.title("PDFRAGAssistant")
3
4st.markdown("""
5**Ứng dụng AI giúp bạn hỏi đáp trực tiếp về nội dung tài liệu PDF bằng tiếng Việt**
6**Cách sử dụng đơn giản:**
71.**Upload PDF**   Chọn file PDF từ máy tính và nhấn "Xử lý PDF"
82.**Đặt câu hỏi**   Nhập câu hỏi về nội dung tài liệu và nhấn câu trả lời ngay lập tức
9---
10""")
```

III.5.2. Tải models

```
1if not st.session_state.models_loaded:
2st.info("Đang tải models...")
3st.session_state.embeddings=load_embeddings()
4st.session_state.llm=load_llm()
5st.session_state.models_loaded=True
6st.success("Models đã sẵn sàng!")
7st.rerun()
```

Đoạn code này kiểm tra và tải models chỉ một lần duy nhất khi ứng dụng được khởi chạy.

III.5.3. UploadvàxửlýPDF

```
1uploaded_file=st.file_uploader("UploadfilePDF",type="pdf")
2if uploaded_fileand st.button("XửlýPDF"):
3withst.spinner("Đangxửlý..."):
4st.session_state.rag_chain,num_chunks=process_pdf(uploaded_file)
5st.success(f"Hoànthành!{num_chunks}chunks")
```

III.5.4. Giaodiệnhỏiđáp

```
1if st.session_state.rag_chain:
2question=st.text_input("Đặtcâu hỏi:")
3if question:
4withst.spinner("Đangtrảlời..."):
5output=st.session_state.rag_chain.invoke(question)
6answer=output.split("Answer:")[1].strip()if "Answer:"in outputelse
    output.strip()
7st.write("**Trảlời:**")
8st.write(answer)
```

Phần này cho phép người dùng đặt câu hỏi và nhận câu trả lời từ RAG system. Câu trả lời được xử lý để loại bỏ phần "Answer:" nếu có trong output của model.

III.6. Chạy ứng dụng và demo từng bước

III.6.1. Khởi chạy ứng dụng Streamlit

Để khởi chạy ứng dụng, ta sử dụng lệnh sau trong terminal:

```
streamlit run app.py
```

Trong đó `app.py` là tên file chứa code ứng dụng của chúng ta.

III.6.2. Bước 1: Tải Embedding Model

Khi ứng dụng được khởi chạy lần đầu tiên, hệ thống sẽ tự động tải embedding model. Giao diện sẽ hiển thị thông báo như trong Hình 9.



Hình 9: Giao diện khi đang tải embedding model

Quá trình này có thể mất vài phút tùy thuộc vào tốc độ internet và cấu hình máy tính. Model `embeddingvietnamese-bi-encoder` sẽ được tải từ HuggingFaceHub.

III.6.3. Bước 2: Tải Large Language Model

Sau khi embedding model được tải xong, hệ thống tiếp tục tải LLM (Vicuna 7B). Đây là bước tốn thời gian nhất do kích thước model lớn (khoảng 14GB). Quá trình này được thể hiện trong Hình 10.

III.6.4. Bước 3: Upload và xử lý PDF

Sau khi model sẵn sàng, người dùng có thể upload file PDF và nhấn "Xử lý PDF". Quá trình xử lý sẽ hiển thị spinner "Đang xử lý..." và khi hoàn thành sẽ thông báo số chunks đã tạo như trong Hình 11. Quá trình này bao gồm: đọc PDF, semantic chunking, tạo embeddings, và xây dựng RAG chain.



Hình10:GiaodiệnkhiđangtảiLargeLanguageModel

III.6.5. Bước4:Hỏidápvới tài liệu

Sau khi xử lý PDF thành công, người dùng có thể đặt câu hỏi và nhận câu trả lời ngay lập tức. Giaodiện sẽ hiển thị "Đang trả lời..." khi xử lý và sau đó hiển thị câu trả lời định dạng như trong Hình12.



Hình11:QuátrìnhuploadvàxửlýPDF



Hình12:GiaođiệnhỏđápvétàiliệuPDF

Người dùng có thể tiếp tục đặt nhiều câu hỏi khác nhau mà không cần xử lý lại PDF.

IV. Câu hỏi trắc nghiệm

1. Tại sao phải sử dụng `@st.cache_resource` cho việc load model trong Streamlit?
- (a) Để tránh reload models mỗi khi người dùng tương tác với giao diện.

(b) Để giảm thời gian phản hồi của ứng dụng.

(c) Để chia sẻ models giữa các session khác nhau.

(d) Cả A và C đều đúng.
2. Trong `SemanticChunker`, `breakpoint_threshold_amount=95` có nghĩa là gì?
- (a) Chia tài liệu thành 95 chunks.

(b) Sử dụng 95% nội dung để tạo chunks.

(c) Chỉ tạo breakpoint khi similarity < 95th percentile.

(d) Minimum chunk size là 95 tokens.
3. Tại sao sử dụng `torch_dtype=torch.bfloat16` thay vì `float32`?
- (a) Tăng tốc độ inference.

(b) Giảm 50% memory usage.

(c) Tương thích tốt hơn với GPU.

(d) Tất cả các lý do trên.
4. Trong `RAGchain`, `RunnablePassthrough()` có vai trò gì?
- (a) Truyền question trực tiếp qua các bước tiếp theo.

(b) Lọc và làm sạch input question.

(c) Chuyển đổi question thành embedding.

(d) Kết hợp question với context.
5. Trong `SemanticChunker`, `buffer_size=1` có ý nghĩa gì?
- (a) Số lượng câu được so sánh khi tìm breakpoint.

(b) Kích thước tối thiểu của mỗi chunk.

(c) Số lượng chunks được xử lý cùng lúc.

(d) Độ dài context window cho embedding.
6. Tại sao phải sử dụng `st.rerun()` sau khi load models xong?
- (a) Để refresh toàn bộ session state.

(b) Để cập nhật UI sau khi thay đổi session state.

(c) Để giải phóng memory đã sử dụng.

(d) Để reset cache của `@st.cache_resource`.

7. Tham số `max_new_tokens=512` có ý nghĩa gì?
- (a) Giới hạn độ dài input context.
 - (b) Số token tối đa model có thể sinh ra.
 - (c) Kích thước vocabulary của model.
 - (d) Số token xử lý song song.
8. Tại sao phải tạo temporary file khi xử lý PDF upload?
- (a) Streamlit không thể đọc trực tiếp file upload.
 - (b) `PyPDFLoader` yêu cầu file path thật.
 - (c) Để tránh conflict với session state.
 - (d) Để backup file trước khi xử lý.
9. Prompt `"rlm/rag-prompt"` từ LangChain Hub có cấu trúc như thế nào?
- (a) Chỉ chứa context và question.
 - (b) Context + Question + Answer format.
 - (c) System prompt + Context + Question.
 - (d) Question + Context + Instructions.
10. Tại sao phải xử lý `output.split("Answer:")[1].strip()`?
- (a) Loại bỏ phần context trong output.
 - (b) Tách phần trả lời từ format prompt.
 - (c) Làm sạch whitespace thừa.
 - (d) Cả B và C đều đúng.

V. Tài liệu tham khảo

[1] L.Zhengetal.,*Judging llm-as-a-judge with mt-bench and chatbot arena*,2023.[Online]. Available:<https://arxiv.org/abs/2306.05685>.

[2] P.Lewisetal.,“Retrieval-augmentedgenerationforknowledge-intensivenlptasks”,in*34th Conference on Neural Information Processing Systems (NeurIPS 2020)*,2020.arXiv:2005.11401 [cs.CL].

[3] Streamlit,*A faster way to build and share data apps*,GitHubrepository,2024.[Online]. Available:<https://github.com/streamlit/streamlit>.

1. **Hint:** Các file code gợi ý có thể được tải tại đây.

2. **Solution:** Các file code cài đặt hoàn chỉnh và phần trả lời nội dung trắc nghiệm có thể được tải tại đây (**Lưu ý:** Sáng thứ 3 khi hết deadline phần project, ad mới copy các nội dung bài giải nêu trên vào đường dẫn).

3. **Kiến thức cần cho project:**

Để hoàn thành và hiểu rõ phần project một cách hiệu quả nhất, các học viên cần nắm rõ các kiến thức được trang bị

- Kiến thức lập trình Python trong các buổi học chính trong Module 1 của khoá học
- Sử dụng công cụ Streamlit trong Project 1.1-Web and Streamlit for Simple Deployment của Module 1
- Thao tác cơ bản với git và github để quản lý code và tương tác với Streamlit trong các buổi học nâng cao vào thứ 5 hoặc thứ 7

4. **Đề xuất phương pháp cải tiến project:**

Project xây dựng chatbot hỏi đáp đơn giản thông qua kiến trúc của RAG. Để tiếp tục cải tiến, tối ưu hệ thống; nhóm tác biên soạn gợi ý một số phương pháp như sau:

- Phương pháp tách đoạn (Chunking): Gợi ý cải tiến phương pháp chia đoạn để tăng tốc độ xử lý và hiệu quả khi chia đoạn
- Mô hình nhúng (Embedding Model): Gợi ý sử dụng hoặc tối ưu các mô hình nhúng từ để có phương pháp biểu diễn từ tốt hơn, tối ưu khả năng truy vấn thông tin, đặc biệt cho ngôn ngữ tiếng việt
- Mô hình ngôn ngữ lớn (LLMs): Bên cạnh mô hình đề xuất, có thể sử dụng những mô hình khác hiệu quả hơn, hoặc sử dụng những kỹ thuật như lượng tử hoá, chắt lọc tri thức,... để tăng tốc độ xử lý, tăng hiệu quả và giảm kích thước của LLMs
- Bộ nhớ dài hạn cho hệ thống: Xây dựng và mở rộng hệ thống lưu trữ những câu hỏi đã được trả lời để tái sử dụng hoặc tận dụng các đáp án có sẵn
- Tri thức cho truy vấn: Bổ sung thêm những dạng tri thức biểu diễn khác cho văn bản hoặc kết hợp thêm các phương pháp tìm kiếm khác nhau như Bm25 kết hợp cosine similarity,...

Trên đây là một số gợi ý, giúp học viên có thể cải tiến thêm hệ thống để đạt hiệu quả tốt hơn về cả tốc độ xử lý và độ chính xác truy vấn, trả lời câu hỏi.

5.Rubric:

Mục	KiếnThức	ĐánhGiá
II.	-KiếnthứcvềsửdụngGoogleColab. -KiếnthứccobảnvềLLMs. -KiếnthứccobảnvềmộtchươngtrìnhRAG.	-NắmđượckháiniệmvềbàitoánRAG. -Hiểudượcnghuyênlýhoạtđộngcủa chươngtrìnhhoidáptàiliệubàihọc AIOsửdụngRAG. -BiếtcáchsửdụngGoogleColabđể triểnkhaimộtyứngdụngvềRAG. -Biếtcáchsửdụngmộtmôhìnhngôn ngữlớntreColab.
III.	- Kiến thức về Streamlit.	-BiếtcáchsửdụngStreamlitđểthiếtkế mộtgiaodiệnwebđơngiảnnhằm phụcvụchodemoRAG.